

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Интеллектуальный анализ данных»
Тема: «РСА»

Выполнил:
Студент 4 курса
Группы ИИ-25
Мархель А.Д.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: научиться применять метод PCA для осуществления визуализации данных.

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование `numpy.linalg.eig` для вычисления собственных значений и собственных векторов и 2. с помощью `sklearn.decomposition.PCA` для непосредственного применения метода PCA – два независимых варианта решения);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки `matplotlib`, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;
4. Оформить отчет по выполненной работе.

Вариант 6

№ варианта	Выборка	Класс
6	seeds.zip	Последняя колонка

Ход работы

Seeds — набор данных о геометрических параметрах зёрен трёх сортов пшеницы (Kama, Rosa, Canadian), полученных по рентгеновским снимкам. Выборка содержит 210 объектов (по 70 на каждый сорт), 7 числовых признаков и метку класса в последней колонке. Файл `seeds_dataset.txt` не имеет заголовка, значения разделены табуляцией. Фрагмент исходных данных:

Area	Perimeter	Compactness	Kernel Length	Kernel Width	Asymmetry	Groove Length	Class
15.26	14.84	0.871	5.763	3.312	2.221	5.22	1
14.88	14.57	0.8811	5.554	3.333	1.018	4.956	1
14.29	14.09	0.905	5.291	3.337	2.699	4.825	1
13.84	13.94	0.8955	5.324	3.379	2.259	4.805	1

Class	Сорт пшеницы	Объектов
1	Kama	70
2	Rosa	70
3	Canadian	70

Признаки: Area — площадь зерна A; Perimeter — периметр P; Compactness — компактность $C = 4\pi A/P^2$; Kernel Length — длина зерна; Kernel Width — ширина зерна; Asymmetry — коэффициент асимметрии; Groove Length — длина бороздки зерна.

Работа выполнена в двух независимых скриптах: pca_manual.py (вручную, numpy.linalg.eig) и pca_sklearn.py (sklearn.decomposition.PCA). Перед применением PCA признаки стандартизованы (нулевое среднее, единичная дисперсия), так как они измерены в разных единицах и имеют разный масштаб. Основная функция ручной реализации:

```
def pca_method(data, n_components):
    cov_matrix = np.cov(data.T)
    V, PC = np.linalg.eig(cov_matrix)
    V, PC = V.real, PC.real
    sort_index = np.argsort(-1 * V)
    V = V[sort_index]
    PC = PC[:, sort_index]
    signs = np.sign(PC[np.argmax(np.abs(PC), axis=0), range(PC.shape[1])])
    PC = PC * signs
    projected = np.dot(PC.T[0:n_components], data.T).T
    return projected, V, PC
```

Реализация с помощью scikit-learn:

```
Xs = StandardScaler().fit_transform(X)
pca2 = PCA(n_components=2)
Z2 = fixed_sign(pca2, pca2.fit_transform(Xs))
pca3 = PCA(n_components=3)
Z3 = fixed_sign(pca3, pca3.fit_transform(Xs))
```

Результат

Ручной PCA (numpy.linalg.eig):

Код программы:

```
from pathlib import Path
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
BASE_DIR = Path(__file__).resolve().parent
DATA_FILE = BASE_DIR / "seeds_dataset.txt"
FIG_DIR = BASE_DIR / "figures"
FIG_DIR.mkdir(exist_ok=True)
```

```
FEATURES = [
    "Area",          # площадь A
    "Perimeter",     # периметр P
    "Compactness",   # компактность  $C = 4*\pi*A/P^2$ 
    "Kernel_Length", # длина зерна
```

```

"Kernel_Width",    # ширина зерна
"Asymmetry",       # коэффициент асимметрии
"Groove_Length",   # длина бороздки зерна
]
CLASS_COL = "Class"
CLASS_NAMES = {1: "Kama", 2: "Rosa", 3: "Canadian"}
CLASS_COLORS = {1: "tab:blue", 2: "tab:red", 3: "tab:green"}
CLASS_MARKERS = {1: "o", 2: "s", 3: "^"}

def load_data():

    df = pd.read_csv(DATA_FILE, sep=r"\s+", header=None,
                     names=FEATURES + [CLASS_COL], engine="python")

    missing = int(df[FEATURES].isna().sum().sum())
    if missing:
        df[FEATURES] = df[FEATURES].fillna(df[FEATURES].median())
        print(f'Загружено объектов: {len(df)}, признаков: {len(FEATURES)}, '
              f'пропусков заменено: {missing}')
        print("Распределение по классам:",
              {CLASS_NAMES[k]: int(v) for k, v in
df[CLASS_COL].value_counts().sort_index().items()})

    X = df[FEATURES].to_numpy(dtype=float)
    y = df[CLASS_COL].to_numpy(dtype=int)
    return X, y

def standardize(X):
    return (X - X.mean(axis=0)) / X.std(axis=0)

def pca_method(data, n_components):
    cov_matrix = np.cov(data.T)
    V, PC = np.linalg.eig(cov_matrix)
    V, PC = V.real, PC.real
    sort_index = np.argsort(-1 * V)
    V = V[sort_index]
    PC = PC[:, sort_index]
    signs = np.sign(PC[np.argmax(np.abs(PC), axis=0), range(PC.shape[1])])
    PC = PC * signs
    projected = np.dot(PC.T[0:n_components], data.T).T
    return projected, V, PC

```

```

def plot_2d(Z, y, V, title, filename):
    total = V.sum()
    fig, ax = plt.subplots(figsize=(8, 6))
    for cls in CLASS_NAMES:
        m = y == cls
        ax.scatter(Z[m, 0], Z[m, 1], c=CLASS_COLORS[cls], marker=CLASS_MARKERS[cls],
                    label=CLASS_NAMES[cls], edgecolors="k", linewidths=0.4, alpha=0.8)
    ax.set_xlabel(f'PC1 ({V[0] / total * 100:.1f}% дисперсии)')
    ax.set_ylabel(f'PC2 ({V[1] / total * 100:.1f}% дисперсии)')
    ax.set_title(title)
    ax.grid(alpha=0.3)
    ax.legend()
    fig.tight_layout()
    fig.savefig(FIG_DIR / filename, dpi=150)

```

```

def plot_3d(Z, y, V, title, filename):
    total = V.sum()
    fig = plt.figure(figsize=(9, 7))
    ax = fig.add_subplot(111, projection="3d")
    for cls in CLASS_NAMES:
        m = y == cls
        ax.scatter(Z[m, 0], Z[m, 1], Z[m, 2], c=CLASS_COLORS[cls],
                    marker=CLASS_MARKERS[cls], label=CLASS_NAMES[cls],
                    edgecolors="k", linewidths=0.3, alpha=0.85, depthshade=False)
    ax.set_xlabel(f'PC1 ({V[0] / total * 100:.1f}%)')
    ax.set_ylabel(f'PC2 ({V[1] / total * 100:.1f}%)')
    ax.set_zlabel(f'PC3 ({V[2] / total * 100:.1f}%)')
    ax.set_title(title)
    ax.view_init(elev=20, azim=-60)
    ax.legend()
    fig.tight_layout()
    fig.savefig(FIG_DIR / filename, dpi=150)

```

```

def main():
    X, y = load_data()
    Xs = standardize(X)

    Z2, V, PC = pca_method(Xs, 2)
    Z3, _, _ = pca_method(Xs, 3)

    print("\nСобственные значения ковариационной матрицы (numpy.linalg.eig):")
    full_info = V.sum()
    for i, v in enumerate(V, 1):
        print(f'  $\lambda_{\{i\}} = \{v:.4f\}$  (доля дисперсии =  $\{v / full\_info * 100:6.2f\}\%$ )')

```

```

print("\nСобственные векторы (столбцы — главные компоненты PC1..PC3):")
print(pd.DataFrame(PC[:, :3], index=FEATURES, columns=["PC1", "PC2", "PC3"]).round(4))

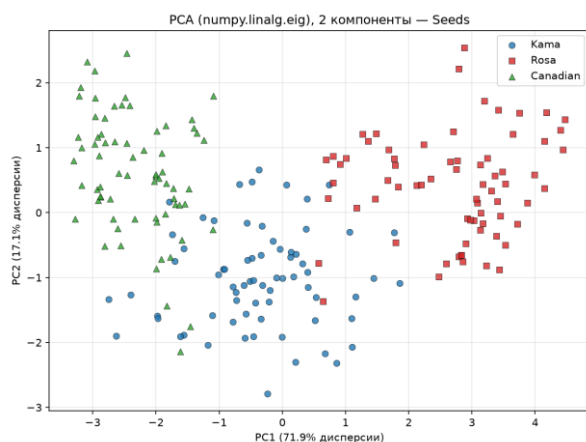
print("\nПотери информации:")
for k in (2, 3):
    reduce_info = V[0:k].sum()
    loss = 100 - reduce_info / full_info * 100
    print(f" [k={k}] сохранено = {reduce_info / full_info * 100:.2f}% | "
          f"потеря = {loss:.2f}%")

plot_2d(Z2, y, V, "PCA (numpy.linalg.eig), 2 компоненты — Seeds", "manual_2d.png")
plot_3d(Z3, y, V, "PCA (numpy.linalg.eig), 3 компоненты — Seeds", "manual_3d.png")
print(f"\nГрафики сохранены в папку: {FIG_DIR}")
plt.show()

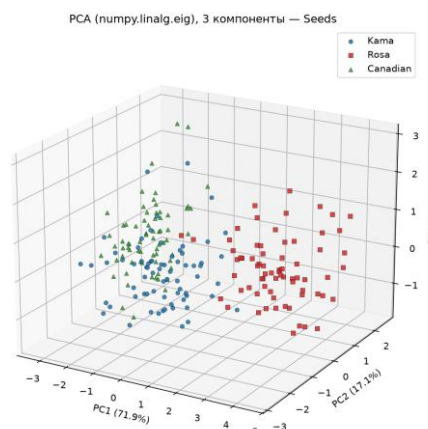
if __name__ == "__main__":
    main()

```

2 главные компоненты



3 главные компоненты



Загружено объектов: 210, признаков: 7, пропусков заменено: 0
 Распределение по классам: {'Kama': 70, 'Rosa': 70, 'Canadian': 70}

Собственные значения ковариационной матрицы (numpy.linalg.eig):

```

λ1 = 5.0553 (доля дисперсии = 71.87%)
λ2 = 1.2033 (доля дисперсии = 17.11%)
λ3 = 0.6812 (доля дисперсии = 9.69%)
λ4 = 0.0687 (доля дисперсии = 0.98%)
λ5 = 0.0188 (доля дисперсии = 0.27%)
λ6 = 0.0054 (доля дисперсии = 0.08%)
λ7 = 0.0008 (доля дисперсии = 0.01%)

```

Собственные векторы (столбцы — главные компоненты PC1..PC3):

	PC1	PC2	PC3
Area	0.4445	0.0266	0.0259
Perimeter	0.4416	0.0840	-0.0598
Compactness	0.2770	-0.5292	0.6297

```
Kernel_Length  0.4236  0.2060 -0.2119
Kernel_Width   0.4328 -0.1167  0.2165
Asymmetry      -0.1187  0.7169  0.6795
Groove_Length  0.3872  0.3772 -0.2139
```

Потери информации:

```
[k=2] сохранено = 88.98% | потеря = 11.02%
[k=3] сохранено = 98.67% | потеря = 1.33%
```

Sklearn PCA:

Код программы:

```
from pathlib import Path
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

```
BASE_DIR = Path(__file__).resolve().parent
DATA_FILE = BASE_DIR / "seeds_dataset.txt"
FIG_DIR = BASE_DIR / "figures"
FIG_DIR.mkdir(exist_ok=True)
```

```
FEATURES = [
    "Area", "Perimeter", "Compactness", "Kernel_Length",
    "Kernel_Width", "Asymmetry", "Groove_Length",
]
CLASS_COL = "Class"
CLASS_NAMES = {1: "Kama", 2: "Rosa", 3: "Canadian"}
CLASS_COLORS = {1: "tab:blue", 2: "tab:red", 3: "tab:green"}
CLASS_MARKERS = {1: "o", 2: "s", 3: "^"}
```

```
def load_data():
    df = pd.read_csv(DATA_FILE, sep=r"\s+", header=None,
                     names=FEATURES + [CLASS_COL], engine="python")
    X = df.drop(columns=[CLASS_COL]) # класс в PCA не участвует
    y = df[CLASS_COL].to_numpy(dtype=int)
    print(f"Загружено объектов: {len(df)}, признаков: {X.shape[1]}, "
          f"пропусков: {int(X.isna().sum().sum())}")
    X = SimpleImputer(strategy="median").fit_transform(X)
    return X, y
```

```
def fixed_sign(pca, Z):
    comps = pca.components_
    signs = np.sign(comps[range(comps.shape[0]), np.argmax(np.abs(comps), axis=1)])
    return Z * signs
```

```

def plot_2d(Z, y, ratio, title, filename):
    fig, ax = plt.subplots(figsize=(8, 6))
    for cls in CLASS_NAMES:
        m = y == cls
        ax.scatter(Z[m, 0], Z[m, 1], c=CLASS_COLORS[cls], marker=CLASS_MARKERS[cls],
                    label=CLASS_NAMES[cls], edgecolors="k", linewidths=0.4, alpha=0.8)
    ax.set_xlabel(f"PC1 ({ratio[0] * 100:.1f}% дисперсии)")
    ax.set_ylabel(f"PC2 ({ratio[1] * 100:.1f}% дисперсии)")
    ax.set_title(title)
    ax.grid(alpha=0.3)
    ax.legend()
    fig.tight_layout()
    fig.savefig(FIG_DIR / filename, dpi=150)

def plot_3d(Z, y, ratio, title, filename):
    fig = plt.figure(figsize=(9, 7))
    ax = fig.add_subplot(111, projection="3d")
    for cls in CLASS_NAMES:
        m = y == cls
        ax.scatter(Z[m, 0], Z[m, 1], Z[m, 2], c=CLASS_COLORS[cls],
                    marker=CLASS_MARKERS[cls], label=CLASS_NAMES[cls],
                    edgecolors="k", linewidths=0.3, alpha=0.85, depthshade=False)
    ax.set_xlabel(f"PC1 ({ratio[0] * 100:.1f}%)")
    ax.set_ylabel(f"PC2 ({ratio[1] * 100:.1f}%)")
    ax.set_zlabel(f"PC3 ({ratio[2] * 100:.1f}%)")
    ax.set_title(title)
    ax.view_init(elev=20, azimuth=-60)
    ax.legend()
    fig.tight_layout()
    fig.savefig(FIG_DIR / filename, dpi=150)

def main():
    X, y = load_data()
    Xs = StandardScaler().fit_transform(X)

    pca2 = PCA(n_components=2)
    Z2 = fixed_sign(pca2, pca2.fit_transform(Xs))
    pca3 = PCA(n_components=3)
    Z3 = fixed_sign(pca3, pca3.fit_transform(Xs))

    # Полный PCA — чтобы вывести все собственные значения.
    pca_full = PCA().fit(Xs)
    V = pca_full.explained_variance_
    ratio = pca_full.explained_variance_ratio_
    print("\nСобственные значения (explained_variance_):")
    for i, (v, r) in enumerate(zip(V, ratio), 1):
        print(f"λ{i} = {v:.4f} (доля дисперсии = {r * 100:6.2f}%)")

    print("\nГлавные компоненты (components_):")
    print(pd.DataFrame(pca3.components_.T, index=FEATURES,

```

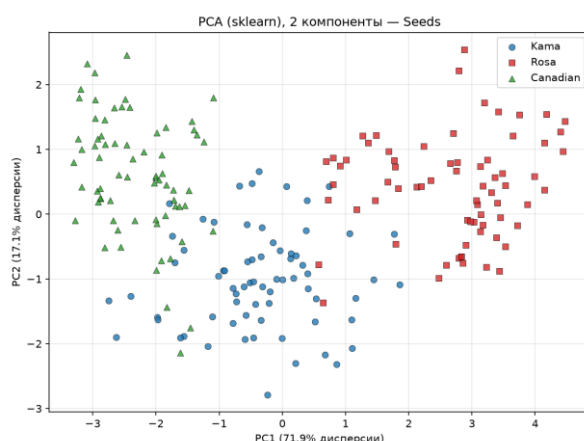
```
columns=["PC1", "PC2", "PC3"]).round(4))

print("\nПотери информации:")
for k, model in ((2, pca2), (3, pca3)):
    kept = model.explained_variance_ratio_.sum() * 100
    print(f" [k={k}] сохранено = {kept:.2f}% | потеря = {100 - kept:.2f}%")

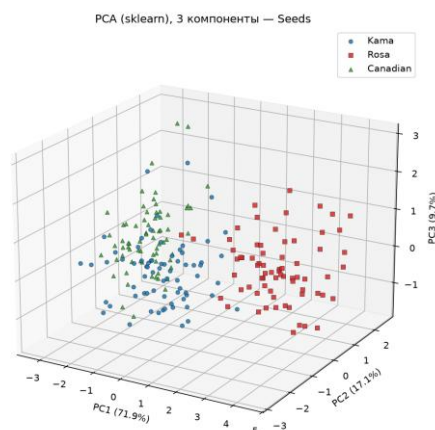
plot_2d(Z2, y, ratio, "PCA (sklearn), 2 компоненты — Seeds", "sklearn_2d.png")
plot_3d(Z3, y, ratio, "PCA (sklearn), 3 компоненты — Seeds", "sklearn_3d.png")
print(f"\nГрафики сохранены в папку: {FIG_DIR}")
plt.show()

if __name__ == "__main__":
    main()
```

2 главные компоненты



3 главные компоненты



Загружено объектов: 210, признаков: 7, пропусков: 0

Собственные значения (explained_variance_):

$\lambda_1 = 5.0553$ (доля дисперсии = 71.87%)
 $\lambda_2 = 1.2033$ (доля дисперсии = 17.11%)
 $\lambda_3 = 0.6812$ (доля дисперсии = 9.69%)
 $\lambda_4 = 0.0687$ (доля дисперсии = 0.98%)
 $\lambda_5 = 0.0188$ (доля дисперсии = 0.27%)
 $\lambda_6 = 0.0054$ (доля дисперсии = 0.08%)
 $\lambda_7 = 0.0008$ (доля дисперсии = 0.01%)

Главные компоненты (components_):

	PC1	PC2	PC3
Area	0.4445	0.0266	0.0259
Perimeter	0.4416	0.0840	-0.0598
Compactness	0.2770	-0.5292	0.6297
Kernel_Length	0.4236	0.2060	-0.2119
Kernel_Width	0.4328	-0.1167	0.2165
Asymmetry	-0.1187	0.7169	0.6795
Groove_Length	0.3872	0.3772	-0.2139

Потери информации:

[k=2] сохранено = 88.98% | потеря = 11.02%
 [k=3] сохранено = 98.67% | потеря = 1.33%

Оба способа дали одинаковые собственные значения, собственные векторы и проекции (максимальное расхождение координат $\approx 3 \cdot 10^{-15}$, т.е. на уровне машинной точности).

Потери информации при проецировании

Потери вычислены по собственным значениям: потеря = $100 - (\lambda_1 + \dots + \lambda_k) / (\lambda_1 + \dots + \lambda_7) \cdot 100\%$.

k (компонент)	Сохранённая дисперсия	Потеря
2	88.98%	11.02%
3	98.67%	1.33%

Дисперсия в выборке Seeds сосредоточена в первых компонентах: $\lambda_1 = 5.06$ (71.87%) — почти три четверти всей информации. Это объясняется сильной корреляцией размерных признаков (площадь, периметр, длина и ширина зерна, длина бороздки), которые фактически описывают одну величину — «размер» зерна. При проекции на 2 компоненты теряется 11.02% информации, на 3 компоненты — лишь 1.33%. Компоненты 4–7 (суммарно 1.33%) практически не несут информации, поэтому семимерные данные без заметных потерь описываются тремя признаками, а для визуализации вполне достаточно двух.

Контрольные вопросы

1. Как была выполнена подготовка датасета и выделение целевого класса?
2. Как ведут себя классы на 2D и 3D графиках PCA для зерен?

1) Подготовка датасета и выделение целевого класса

- Файл `seeds_dataset.txt` (из архива `seeds.zip`) загружен с помощью `pandas.read_csv` с параметром `header=None`, так как заголовка в файле нет; колонкам присвоены имена `Area`, `Perimeter`, `Compactness`, `Kernel_Length`, `Kernel_Width`, `Asymmetry`, `Groove_Length`, `Class`.
- В файле значения разделены табуляцией, но в ряде строк встречается двойная табуляция (например, в 8-й строке). При `sep='\t'` `pandas` получил бы лишние пустые колонки и сдвиг значений, поэтому использован разделитель `sep=r'\s+'` (любое количество пробельных символов). В результате получено 210 строк $\times$ 8 колонок.
- Выполнена проверка на пропуски (`isna`): пропущенных значений не обнаружено. На случай их появления предусмотрена стратегия замещения медианой столбца (`fillna(median)` в ручном варианте и `SimpleImputer(strategy='median')` в варианте со `sklearn`).
- Целевой класс — последняя (8-я) колонка `Class` со значениями 1 — Кама, 2 — Rosa, 3 — Canadian (по 70 объектов, классы сбалансированы). Он выделен в отдельный вектор `y` и удалён из матрицы признаков (`df.drop(columns=['Class'])`), т.е. используется только для раскраски точек на графиках и в вычислении PCA не участвует.
- Оставшиеся 7 признаков стандартизованы: из каждого вычтено среднее и результат поделён на стандартное отклонение. Это необходимо, так как признаки имеют разный масштаб (площадь 10.59–21.18, компактность 0.808–0.918), и без стандартизации PCA был бы определён в основном признаками с наибольшей дисперсией.

2) Поведение классов на 2D и 3D графиках PCA

Первая главная компонента PC1 (71.9% дисперсии) имеет почти одинаковые положительные веса у размерных признаков (Area 0.44, Perimeter 0.44, Kernel Width 0.43, Kernel Length 0.42, Groove Length 0.39), т.е. это обобщённый «размер» зерна. Вторая компонента PC2 (17.1%) в основном определяется коэффициентом асимметрии (0.72) и компактностью (−0.53) — это «форма» зерна.

На 2D-графике классы располагаются вдоль оси PC1 в порядке роста размера зерна: слева — Canadian (самые мелкие зёрна, среднее PC1 ≈ -2.32), в центре — Кама (≈ -0.36), справа — Rosa (самые крупные, ≈ 2.68). Сорт Rosa отделён от остальных практически полностью. Кама и Canadian частично перекрываются в области PC1 $\approx -2 \dots -1$, но разводятся по оси PC2: у Кама значения в основном отрицательные (среднее ≈ -1.02), у Canadian — положительные (≈ 0.61), т.е. зёрна Canadian более асимметричны и менее компактны. Точки Кама занимают промежуточное положение и отчасти соприкасаются и с Rosa, и с Canadian. Классификация по ближайшему центру класса в плоскости PC1–PC2 даёт 92.9% верных отнесений.

На 3D-графике добавляется компонента PC3 (9.7%), которая связана с асимметрией и компактностью одновременно (0.68 и 0.63). Средние значения PC3 у всех трёх классов близки к нулю (−0.03, −0.01, 0.04), т.е. третья компонента описывает в основном разброс внутри классов, а не различия между ними. Взаимное расположение классов остаётся тем же, что и на 2D-графике: Rosa обособлена, Кама и Canadian граничат друг с другом; точность классификации по ближайшему центру в 3D практически не меняется (92.4%). Таким образом, для визуального разделения сортов пшеницы достаточно двух главных компонент.

Вывод: научилась применять метод PCA для осуществления визуализации данных. Метод реализован двумя независимыми способами (numpy.linalg.eig и sklearn.decomposition.PCA), результаты полностью совпали. Для выборки Seeds проекция на 2 главные компоненты сохраняет 88.98% информации (потеря 11.02%), на 3 компоненты — 98.67% (потеря 1.33%); на графиках три сорта пшеницы образуют хорошо различимые группы.